12-1 fastapi是异步Web框架

从本教程开篇，我们就说FastAPI这个web框架是异步框架，那它到底是如何体现异步的呢？

想要学习使如何使用FastAPI的异步功能，那就必须要先了解什么是异步，什么是asyncio、async/await

【基础补充】

关于异步编程、协程实行的异步编程的基础知识，大家参考学习如下两个教程：

- Python进阶教程第7章（https://www.51zxw.net/List.aspx?cid=1072#!fenye=3）
- Python asyncio异步编程教程（https://www.51zxw.net/List.aspx?cid=1054）
- 这两个课程很重要，他们是学习掌握FastAPI异步原理的核心基础知识点。

【重要结论】

本质上，**实现异步的方式有三种：多进程、多线程和协程，FastAPI实现异步使用了多线程(线程池)和协程的方式。**

- 当我们写普通形式的代码时：即使用 `def` 定义路径函数，FastAPI内部帮我们使用多线程（线程池）实现异步并发
- 当我们写async形式的代码时：即使用 `async def`定义路径函数，FastAPI内部使用协程的方式实现异步并发。
- 在一个项目中，我们可以同时普通函数定义的路径函数（api），和async def 形式定义的路径函数（api）
- 但是要记得，在async def 形式定义的函数内，不能使用同步的代码，否则接口的响应速度极慢。

12-2 比较同步和异步代码

在FastAPI中可以使用普通函数定义的接口，也可以使用async def 实行定义的接口。但是使用是需要注意，否则会导致程序极慢。

示例1：普通函数形式定义的接口，会按照多线程（线程池）的方式异步执行

- 比如这个接口，使用`time.sleep(5)`模拟耗时5s
- 当两个客户端同时访问该接口时，服务端就会收到两个请求，因为FastAPI内部使用多线程的方式来处理，一个请求会被一个线程来处理，两个线程几乎同时执行。于是5秒后，两个请求都得到了响应。

```python
import time
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def index():
    time.sleep(5)
    return "index"
```

示例2：async def 形式的定义的接口，使用单线程协程的形式异步执行

- 因为使用了协程的方式，当有两个客户端发请求时，单线程内代码块级别的切换，最终5秒后两个客户端都会得到响应。
- 注意：协程时不能使用同步阻塞的time模块，需要使用asyncio.sleep()。又因为它是协程对象，所以需要使用`await`才能被执行。
- 另外，`await`必须使用在`async`定义的函数内，否则报错。
- 同时注意，在async def 内部不能使用同步模块，否则就会编程单线程同步执行的方式。

```python
import asyncio
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
async def index():
    await asyncio.sleep(5)
    return "index"
```

示例3：在协程函数内错误使用同步模块

- 因为使用async def定义的函数，所以FadtAPI内部使用单线程的协程方式运行代码，所以当两个客户端发请求时，服务端只有一个线程处理两个请求。
- 但因为是同步阻塞5秒，所以只能等第一个请求5秒后处理完，才能开始执行第二个请求。所以第二个请求等待了10秒才能得到响应。

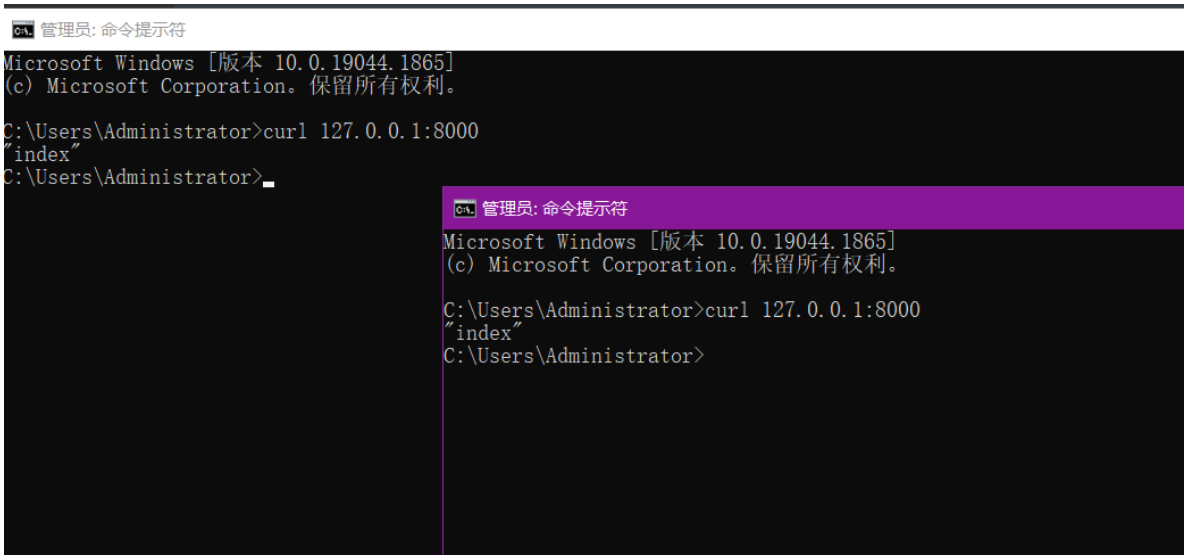
```python
import time
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
async def index():
    time.sleep(5)        # 不能在async def 函数内使用同步阻塞模块。
    return "index"
```

【注意】

- 演示上述代码时，不要在一个浏览器上开两个tab页，可能无法演示出效果。这是因为有些浏览器有设置，一个host之后一个连接，所以打开的两个tab也通用一个和服务端通信的连接，此时的效果就是第一个请求得到响应后才开始第二个响应。
- 你可以使用两个不同的浏览器演示示例；或者开两个cmd窗口，使用 curl命令来发请求。



12-3 同步异步如何选择

FastAPI非常灵活，支持你写普通的函数，也支持你写async def 形式的函数，那到底该如何选择？

下面有几个原则，可以帮助大家做选择：

- 你需要使用一个普通的，内部有IO等待的第三方库，此时使用 `def`

```
@app.get('/')
def results():
    results = some_library()
    return results
```

- 你需要使用的一个第三方库，需要使用 `await`调用时，此时使用 `async def`

```
@app.get('/')
async def read_results():
    results = await some_library()
    return results
```

- 你自己编写的工具库或者第三方工具库，代码执行过程中不涉及IO等待，此时直接调用即可，不用 `await`
- 当你的函数内部只要有一处使用 `await`， 该函数必须使用 `async def`定义

```
@app.get('/')
async def read_results():
    results1 = some_library1()
    results2 = await some_library()
    return results
```

- 当你对协程/async/await/asyncio这些概念不清楚的时候，就使用普通函数。
- 你需要使用的一个第三方库是同步库，但是你需要它支持协程异步，那需要自己使用线程池的方式运行，参考视频：
    - 线程池和协程混合实现并发案例

12-4 fastapi集成aiohttp

传统python代码中发请求（如爬虫）我们一般使用requests模块，但是这个模块是同步阻塞的。

所以在异步asyncio体系中，我们不再使用requests模块，一般会使用异步的 `aiohttp` 模块。（httpx）

- 使用前现在安装

```
pip3 install aiohttp
```

- 简介：异步支持客户端和服务端，功能强大。https://docs.aiohttp.org/en/stable/

示例1：基本使用

```
import asyncio
import aiohttp

# 协程函数
async def aiohttp_demo():
    # 获取一个连接session
    async with aiohttp.ClientSession() as session:
        # 基于连接发送一个get请求并获取像一个response
        async with session.get('http://www.baidu.com') as response:
            # 从response中获取响应的各种结果
            # 因为基于上下文管理器，所以出自动关闭连接
            print("Status:", response.status)
            print("Content-type:", response.headers['content-type'])

            html = await response.text()
            print("Body:", html[:15], "...")


# 获取事件循环，在事件循环中执行协程函数
# loop = asyncio.get_event_loop()
```

```
# loop.run_until_complete(aiohttp_demo())


# 上面两行代码的简写，Python3.7以后的版本才可以使用
asyncio.run(aiohttp_demo())
```

示例2：在fastapi中使用

```
import aiohttp
from fastapi import FastAPI


app = FastAPI(title="使用aiohttp")


@app.get("/")
async def baidu_index():
    async with aiohttp.ClientSession() as session:
        async with session.get('http://www.baidu.com') as response:
            return {
                "status": response.status,
                "content-type": response.headers['content-type'],
                "body": await response.text()
            }
```

- 或者将爬取百度页面的代码封装成一个协程函数，在接口中直接调用该协程函数即可

```
import aiohttp
from fastapi import FastAPI


app = FastAPI(title="使用aiohttp")


async def baidu_html():
    async with aiohttp.ClientSession() as session:
        async with session.get('http://www.baidu.com') as response:
            return {
                "status": response.status,
                "content-type": response.headers['content-type'],
                "body": await response.text()
            }


@app.get("/")
async def index():
    return await baidu_html()       # 需要使用await 才能执行baidu_html 这个协程函数
```

12-5 fastapi集成aiomysql

在python中操作mysql我们通常使用pymysql作为数据库驱动，但是在异步的世界中我们使用aiomysql当驱动。

官网：https://aiomysql.readthedocs.io/en/latest/index.html

- 安装`aiomysql`

```
pip3 install aiomysql
```

示例1：aiomysql简单使用

```
import aiomysql
from aiomysql.cursors import DictCursor

from fastapi import FastAPI


app = FastAPI(title="使用aiomysql")


async def aiomysql_demo():
    # 获取连接对象
    conn = await aiomysql.connect(
        host="127.0.0.1",
        port=3306,
        user="root",
        password="12345",
        db="db",
        cursorclass=DictCursor      # 返回字典格式的数据
    )
    # 创建游标
    cur = await conn.cursor()
    # 执行SQL
    await cur.execute("SELECT * from users;")
    # 获取SQL结果
    result = await cur.fetchall()
    # 关闭CURSOR
    await cur.close()
    # 关闭连接
    conn.close()

    return result


@app.get("/")
async def index():
    return await aiomysql_demo()
```

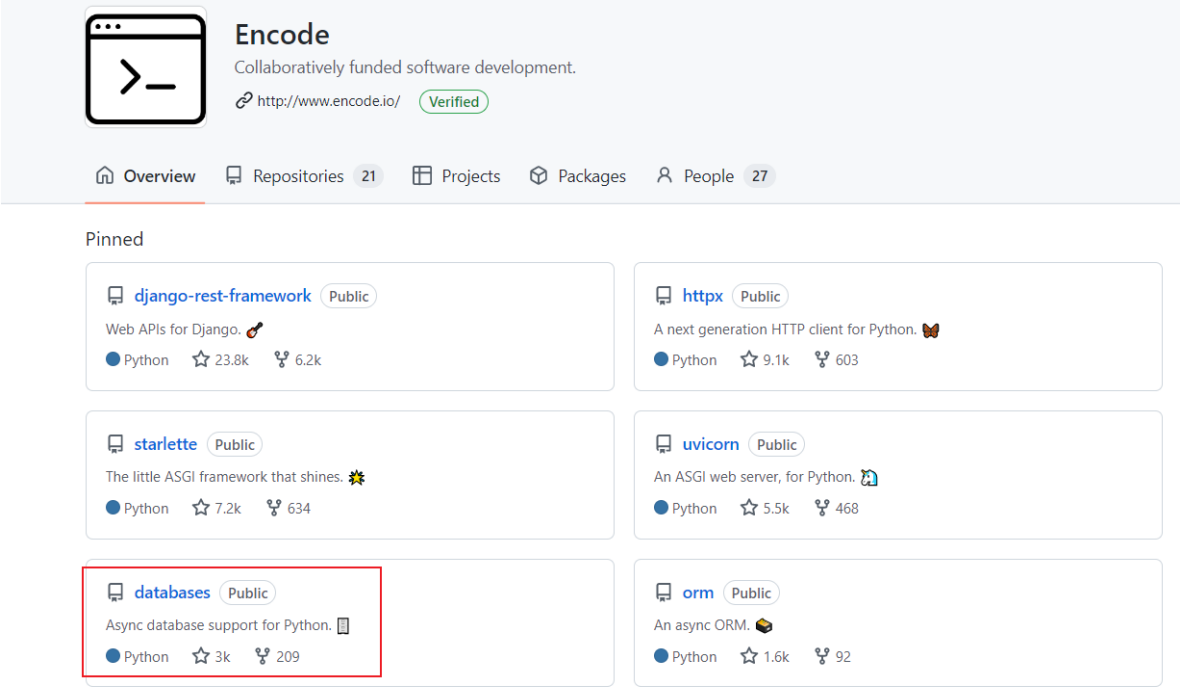
12-6 fastapi集成databases

在异步世界中操作数据库，比如MySQL，我们需要只用 aiomysql，且需要自己手写SQL语句。

使用其他类型的数据库，比如PostgreSQL，则需要基于asyncpg或aiopg。

那样的就有一个问题，当我们的应用需要换一个数据库时，就需要调整基础代码，使用不灵活。

此时出现了一个工具，它封装了不同类型的数据库，我们只需要在使用它提供的接口操作数据库就行了，而不用关心底层的数据库驱动，它就是encode出品的 `databases`。



官网简介

- Databases gives you simple asyncio support for a range of databases.
- It allows you to make queries using the powerful SQLAlchemy Core expression language, and provides support for PostgreSQL, MySQL, and SQLite.
- Databases is suitable for integrating against any async Web framework, such as Starlette, Sanic, Responder, Quart, aiohttp, Tornado, or FastAPI.
- Documentation: https://www.encode.io/databases/
- Requirements: Python 3.7+
- 下载：`pip install databases`

示例：在fastapi中使用databases操作MySQL（依赖aiomysql）

```
from fastapi import FastAPI
from databases import Database


app = FastAPI(title="使用databases")
```

```python
async def databases_demo():
    # 实例化一个db连接并建立连接
    database = Database('mysql://root:12345@localhost:3306/db')
    await database.connect()

    # Run a database query.
    query = "SELECT * FROM users"
    rows = await database.fetch_all(query=query)
    return rows


@app.get("/")
async def index():
    return await databases_demo()
```

12-7 fastapi集成乌龟ORM

前面在第8章，我们给大家介绍了在同步代码中，想要使用ORM操作数据，使用了SQLAlchemy，

同样的在基于协程的异步代码中，操作数据库时也可以使用ORM，但此时就不能在使用SQLAlchemy（因为它不支持异步）

asyncio世界中，我们也有可以选择的ORM，比如： tortoise-orm 翻译过来就乌龟ORM。

乌龟ORM简介

- 官网：https://tortoise-orm.readthedocs.io/en/latest/index.html
- Tortoise ORM is an easy-to-use `asyncio` ORM *(Object Relational Mapper)* inspired by Django.
- Tortoise ORM was build with relations in mind and admiration for the excellent and popular Django ORM.
- Tortoise ORM is supported on CPython >= 3.7 for SQLite, MySQL and PostgreSQL.
- 下载安装：`pip3 install tortoise-orm`

示例：fastapi简单集成乌龟ORM

```python
# main.py

from fastapi import FastAPI
from tortoise import fields
from tortoise.models import Model
from tortoise.contrib.fastapi import register_tortoise
```

```python
app = FastAPI(title="使用tortoise orm")


# 定义模型表
class User(Model):
    id = fields.IntField(pk=True)
    username = fields.CharField(max_length=255)
    password = fields.CharField(max_length=255)
    email = fields.CharField(max_length=255)

    class Meta:
        table = "users"     # 表示这个表对应数据库中的表名


# 使用register_tortoise 注册数据库信息
register_tortoise(
    app,
    db_url="mysql://root:12345@127.0.0.1:3306/db",
    modules={"models": ["main"]},   # 指定模型表所在的文本，"main" 表示mian.py中定义了User模型表
)


@app.get("/")
async def index():
    user = await User.filter(username="liuxu").first()
    user.email = "1111"
    await user.save()

    # 常用的CRUD方法
    # fake_user = await User.create(username="111", password="111", email="111")
    # await User.filter(id=fake_user.id).update(username="Updated name")
    # await User.filter(id=1).delete()

    return user
```